



UNIVERSIDADE EDUARDO MONDLANE
FACULDADE DE ENGENHARIA
DEPARTAMENTO DE ENGENHARIA ELECTROTECNICA
ENGENHARIA INFORMATICA
Base de Dados II
2 ANO - LABORAL

Laboratorio 01: Análise de Volume de Transacções

DISCENTE:

Nhamuave, Nicoleta Dito

DOCENTES:

- Helton Jossias
- Khalid Bapu

Maputo, 4 de Setembro de 2026

1. Medir transacções por minuto

```
MariaDB [companhia]> SELECT DATE_FORMAT(data_hora, '%Y-%m-%d %H:%i:00') AS minuto, COUNT(*) AS total_transacoes FROM transacoes_companhia GROUP BY DATE_FORMAT(data_hora, '%Y-%m-%d %H:%i:00') ORDER BY total_transacoes DESC LIMIT 10;
```

minuto	total_transacoes
2026-01-10 16:04:00	9
2026-01-24 02:09:00	8
2026-01-16 02:30:00	8
2026-01-02 06:10:00	8
2026-01-12 13:07:00	7
2026-01-27 13:36:00	7
2026-01-26 20:43:00	7
2026-01-24 13:57:00	7
2026-01-13 10:22:00	7
2026-01-22 11:11:00	7

10 rows in set (0.189 sec)

```
MariaDB [companhia]> |
```

2. Separar leituras e escritas

```
MariaDB [companhia]> SELECT tipo_operacao, COUNT(*) AS total, ROUND(COUNT(*) * 100 / (SELECT COUNT(*) FROM transacoes_companhia), 2) AS percentagem FROM transacoes_companhia GROUP BY tipo_operacao;
```

tipo_operacao	total	percentagem
LEITURA	32506	65.01
ESCRITA	17494	34.99

2 rows in set (0.061 sec)

```
MariaDB [companhia]> |
```

3. Identificar horários de pico

```
MariaDB [companhia]> SELECT HOUR(data_hora) AS hora, COUNT(*) AS total_transacoes FROM transacoes_companhia GROUP BY HOUR(data_hora) ORDER BY total_transacoes DESC;
```

hora	total_transacoes
5	2187
11	2165
13	2142
21	2118
1	2118
2	2117
14	2098
22	2096
12	2092
9	2086
7	2086
16	2086
6	2085
19	2082
3	2077
17	2067
20	2064
8	2056
15	2056
0	2054
18	2045
10	2041
4	2017
23	1965

24 rows in set (0.031 sec)

```
MariaDB [companhia]> |
```

4. Calcular latência média

```
MariaDB [companhia]> SELECT tipo_operacao, AVG(TIMESTAMPDIFF(SECOND, inicio_execucao, fim_execucao)) AS latencia_media_segundos, MAX(TIMESTAMPDIFF(SECOND, inicio_execucao, fim_execucao)) AS maior_latencia FROM transacoes_companhia GROUP BY tipo_operacao;
```

tipo_operacao	latencia_media_segundos	maior_latencia
LEITURA	5.4873	10
ESCRITA	5.4818	10

2 rows in set (0.078 sec)

```
MariaDB [companhia]> |
```

5. Relatório de throughput

```
MariaDB [companhia]> SELECT DATE_FORMAT(data_hora, '%Y-%m-%d %H:%i:00') AS minuto, COUNT(*) AS transacoes_concluidas, ROUND(COUNT(*) / 60, 2) AS throughput_por_segundo FROM transacoes_companhia WHERE estado = 'CONCLUIDA' GROUP BY DATE_FORMAT(data_hora, '%Y-%m-%d %H:%i:00') ORDER BY throughput_por_segundo DESC LIMIT 10;
```

minuto	transacoes_concluidas	throughput_por_segundo
2026-01-10 16:04:00	9	0.15
2026-01-16 02:30:00	8	0.13
2026-01-24 02:09:00	8	0.13
2026-01-02 06:10:00	8	0.13
2026-01-26 20:43:00	7	0.12
2026-01-12 13:07:00	7	0.12
2026-01-22 11:11:00	7	0.12
2026-01-13 10:22:00	7	0.12
2026-01-27 13:36:00	7	0.12
2026-01-24 13:57:00	7	0.12

10 rows in set (0.180 sec)

```
MariaDB [companhia]>
```

Exercícios

1. Identifique os três horários com maior carga numa semana.

```
MariaDB [companhia]> SELECT  
-> HOUR(data_hora) AS hora_do_dia,  
-> COUNT(*) AS total_transacoes  
-> FROM transacoes_companhia  
-> GROUP BY HOUR(data_hora)  
-> ORDER BY total_transacoes DESC  
-> LIMIT 3;
```

hora_do_dia	total_transacoes
5	2187
11	2165
13	2142

3 rows in set (0.026 sec)

```
MariaDB [companhia]>
```

2. Comparação entre Sistema Read-Heavy (80% Leituras) e Write-Heavy (70% Escritas)

Característica	Sistema Read-Heavy (80% Leituras / 20% Escritas)	Sistema Write-Heavy (70% Escritas / 30% Leituras)
Uso de Índices	Intensivo: Utiliza múltiplos índices (simples e compostos) para agilizar a localização de dados em consultas SELECT.	Mínimo: Limita o uso de índices, pois cada operação de escrita exige atualizar também as estruturas de índices no disco.

Característica	Sistema Read-Heavy (80% Leituras / 20% Escritas)	Sistema Write-Heavy (70% Escritas / 30% Leituras)
Concorrência e Bloqueios	Baixo impacto: Leituras em paralelo raramente geram conflitos graves ou bloqueiam o acesso aos dados.	Alto risco de bloqueios: Requer transacções muito curtas e controlo de isolamento para evitar filas de espera e <i>deadlocks</i> .
Estrutura de Dados	Beneficia da Desnormalização e tabelas resumo para eliminar a necessidade de múltiplos JOINS pesados.	Beneficia da Normalização estrita para eliminar redundâncias e reduzir a quantidade de dados a gravar por registo.
Estratégia de Performance	Uso de <i>caching</i> em memória para reaproveitar os resultados de consultas frequentes.	Foco em <i>commits</i> otimizados, transacções rápidas e minimização do tempo de retenção de <i>locks</i> .